
EFFICIENT AND PRIVACY AWARE EDGE CLOUD COLLABORATIVE INFERENCE FOR LARGE LANGUAGE MODELS

Cheng Li Jiexiong Liu Yixuan Chen Yi Li
KunlunMeta

ABSTRACT

Large language model (LLM) inference on user-facing devices must balance interactive latency, limited local compute resources, and privacy exposure from user inputs and dialogue histories. Pure cloud inference provides abundant compute but exposes raw prompts and long-context identifiers to a remote service, whereas pure on-device inference is often infeasible for consumer CPUs, mobile GPUs, and embedded edge devices. This paper presents a privacy-aware edge-cloud collaborative inference framework derived from an endpoint-authorized KV-cache workflow. The endpoint performs input preprocessing, embedding computation, device-adaptive feature preparation, KV-cache authorization, speculative decoding, and a low-dimensional component of the language-model head. The cloud executes authorized decoder-layer inference, maintains the KV cache, validates speculative tokens, and computes the high-dimensional component of the vocabulary projection. The endpoint then fuses partial logits, applies language-adaptive vocabulary masking, and samples the next token. To reduce practical exposure during collaboration, tensor payloads and clipped logits are quantized and protected with AES-GCM in transit, while LoRA-adapted lightweight modules, draft-module parameters, and cache-access decisions remain local to the endpoint. The framework supports CPU-only devices through block-wise streaming, GPU-equipped devices through local batching and sliced lightweight modules, and embedded devices through quantized ONNX Runtime deployment. We implement a prototype and evaluate it across heterogeneous endpoint profiles under controlled network conditions. The proposed framework reduces per-token latency by up to 46.1% over naive split inference and reduces language-specific downlink payloads by up to 67.4% compared with full-logits transfer, while maintaining task-level metrics within a small margin of the full cloud target model in our prototype evaluation.

Keywords Large Language Models · Edge-Cloud Inference · Privacy-Aware Inference · KV Cache · Speculative Decoding · LoRA · ONNX Runtime

1 Introduction

Large language models (LLMs) have become a core component of interactive applications such as personal assistants, enterprise copilots, industrial monitoring interfaces, and in-vehicle dialogue systems. These applications increasingly require LLM inference to operate close to end users, where inputs and dialogue histories may contain sensitive information and where interactive latency strongly affects usability. However, deploying the full model locally is often impractical. Pure CPU devices may lack sufficient matrix-computation throughput; GPU-equipped endpoints may have limited memory and must support multiple concurrent sessions; and embedded edge devices require cross-platform runtime compatibility and aggressive model compression.

A common engineering solution is to offload inference to the cloud. Cloud-only serving, however, requires transmitting user inputs, history identifiers, or long-context state to a remote server. This creates privacy risks and increases communication overhead. Existing split-inference systems can reduce local computation, but they often lack fine-grained control of the KV cache, device-specific adaptation for CPU/GPU/edge endpoints, and a combined protection strategy for both conventional network security and hidden-state leakage.

适用于大语言模型的高效且具有隐私意识的边缘云协作推理

成李杰雄刘以轩陈毅李昆仑Meta

抽象的

面向用户的设备上的大语言模型 (LLM) 推理必须平衡交互延迟、有限的本地计算资源以及用户输入和对话历史记录的隐私暴露。纯云推理可提供丰富的计算，但会向远程服务公开原始提示和长上下文标识符，而纯设备上推理对于消费类 CPU、移动 GPU 和嵌入式边缘设备通常不可行。本文提出了一种源自端点授权的 KV 缓存工作流程的隐私感知边缘-云协作推理框架。端点执行输入预处理、嵌入计算、设备自适应特征准备、KV 缓存授权、推测性解码和语言模型头的低维组件。云执行授权的解码器层推理、维护 KV 缓存、验证推测令牌并计算词汇投影的高维组件。然后，端点融合部分逻辑，应用语言自适应词汇屏蔽，并对下一个标记进行采样。为了减少协作期间的实际暴露，张量有效负载和剪裁的 logits 在传输过程中通过 AES-GCM 进行量化和保护，而 LoRA 适应的轻量级模块、草稿模块参数和缓存访问决策仍保留在端点本地。该框架通过块方式流支持仅使用 CPU 的设备，通过本地批处理和切片轻量级模块支持配备 GPU 的设备，以及通过量化 ONNX 运行时部署支持嵌入式设备。我们实现了一个原型，并在受控网络条件下跨异构端点配置文件对其进行评估。与简单的分割推理相比，所提出的框架将每个令牌的延迟减少了高达 46.1%，与全 logits 传输相比，将特定于语言的下行链路负载减少了高达 67.4%，同时在我们的原型评估中将任务级指标保持在完整云目标模型的一小部分范围内。

Keywords 大语言模型 · 边缘云推理 · 隐私感知推理 · KV 缓存 · 推测解码 · LoRA · ONNX 运行时

1 简介

大语言模型 (LLM) 已成为交互式应用程序的核心组件，例如个人助理、企业副驾驶、工业监控界面和车载对话系统。这些应用程序越来越需要 LLM 推理来靠近最终用户进行操作，其中输入和对话历史可能包含敏感信息，并且交互延迟会严重影响可用性。然而，在本地部署完整的模型通常是不切实际的。纯 CPU 设备可能缺乏足够的矩阵计算吞吐量；配备 GPU 的端点可能内存有限，并且必须支持多个并发会话；嵌入式边缘设备需要跨平台运行时兼容性和积极的模型压缩。

常见的工程解决方案是将推理卸载到云端。然而，纯云服务需要将用户输入、历史标识符或长上下文状态传输到远程服务器。这会带来隐私风险并增加通信开销。现有的分割推理系统可以减少本地计算，但它们通常缺乏对 KV 缓存的细粒度控制、针对 CPU/GPU/边缘端点的设备特定适配，以及针对传统网络安全和隐藏状态泄漏的组合保护策略。

This paper proposes an edge-cloud collaborative inference architecture for LLMs. The central idea is to keep privacy-sensitive decisions and lightweight computation on the endpoint, while moving heavyweight decoder computation to the cloud under endpoint-controlled cache authorization. The endpoint generates embeddings, configures cache usage, computes a lightweight local projection, performs speculative decoding, and fuses cloud-returned partial logits. The cloud receives feature tensors rather than raw text, performs authorized decoder inference, computes the remaining high-dimensional projection, and verifies speculative predictions. Communication is reduced through language-adaptive vocabulary pruning and device-adaptive tensor transmission.

The main contributions are as follows:

- We present a collaborative LLM inference workflow in which the endpoint controls KV-cache usage and avoids transmitting raw prompts, plaintext history identifiers, or complete output distributions. Unlike conventional split inference, the cache policy is explicitly controlled by the privacy-sensitive endpoint.
- We introduce a distributed vocabulary-projection design that splits the classifier head by hidden-state dimensions, enabling the endpoint to compute a low-dimensional local component while the cloud computes the high-dimensional component without returning a complete logits distribution when language-restricted generation is enabled.
- We integrate speculative decoding with endpoint-cloud verification so that accepted draft tokens can reduce repeated interaction rounds and cloud-side decoding work.
- We describe a heterogeneous deployment strategy for pure CPU, GPU-equipped, and edge devices, including block-wise streaming, model slicing, tensor parallelism, and ONNX Runtime execution.
- We combine AES-GCM transport protection, endpoint-local LoRA modules, and explicit cache authorization to reduce communication-level and hidden-state exposure.

2 Related Work

2.1 Efficient LLM Serving

Modern LLM serving systems rely heavily on batching, memory-efficient attention, and KV-cache management. PagedAttention and related systems improve serving throughput by reducing memory fragmentation and efficiently reusing key-value states [1]. Orthogonal systems work improves attention kernels, continuous batching, and distributed inference execution through FlashAttention, FlashAttention-2, ORCA, and DeepSpeed-Inference [2, 3, 4, 5]. These techniques are primarily designed for datacenter serving, whereas our setting gives the endpoint explicit authority over whether and how much historical KV cache may be used and how much information is exposed during each generation step.

2.2 Split and Edge-Cloud Inference

Split computing moves different parts of a neural network across the endpoint and the cloud. Earlier collaborative-intelligence systems, such as Neurosurgeon, studied layer placement between mobile devices and cloud servers [6]. Split-computing surveys and layer-partitioning systems further characterize latency, bandwidth, and heterogeneous-device trade-offs [7, 8, 9]. Recent LLM-oriented edge-cloud approaches study hybrid small/large models, collaborative edge inference, and split speculative decoding [10, 11, 12, 13, 14, 15]. For LLMs, the large decoder stack, the vocabulary projection, and the KV cache introduce new bottlenecks. Our framework therefore splits not only model layers but also cache access, vocabulary projection, speculative decoding, and device-specific transmission logic, with privacy exposure as a first-class design objective.

2.3 Speculative Decoding

Speculative decoding accelerates autoregressive generation by using a smaller draft model to propose future tokens, which are then verified by a larger target model [16, 17]. Later work extends this idea with tree-based verification, hardware-aware draft trees, multiple decoding heads, lookahead decoding, knowledge-distilled draft models, and self-speculative decoding [18, 19, 20, 21, 22, 23, 24, 25]. In the proposed collaborative framework, the endpoint hosts a lightweight speculative module. The cloud validates draft tokens and updates the KV cache, allowing accepted tokens to reduce interaction rounds without exposing the full endpoint-side module parameters.

本文提出了一种适用于大语言模型的边缘-云协作推理架构。其中心思想是在端点上保留隐私敏感决策和轻量级计算，同时在端点控制的缓存授权下将重量级解码器计算移至云端。端点生成嵌入、配置缓存使用、计算轻量级本地投影、执行推测解码并融合云返回的部分逻辑。云接收特征张量而不是原始文本，执行授权的解码器推理，计算剩余的高维投影，并验证推测性预测。通过语言自适应词汇修剪和设备自适应张量传输减少了通信。

主要贡献如下：

- 我们提出了一种协作式 LLM 推理工作流程，其中端点控制 KV 缓存的使用并避免传输原始提示、纯文本历史标识符或完整的输出分布。与传统的分割推理不同，缓存策略由隐私敏感端点显式控制。
- 我们引入了一种分布式词汇投影设计，该设计通过隐藏状态维度分割分类器头，使端点能够计算低维局部组件，而云计算高维组件，而在启用语言限制生成时不会返回完整的逻辑分布。
- 我们将推测性解码与端点云验证相结合，以便接受的草稿令牌可以减少重复的交互轮次和云端解码工作。
- 我们描述了针对纯 CPU、配备 GPU 和边缘设备的异构部署策略，包括分块流、模型切片、张量并行和 ONNX 运行时执行。
- 我们结合了 AES-GCM 传输保护、端点本地 LoRA 模块和显式缓存授权，以减少通信级别和隐藏状态暴露。

2 相关工作

2.1 高效的大语言模型服务

现代 LLM 服务系统严重依赖批处理、内存高效注意力和 KV 缓存管理。PagedAttention 和相关系统通过减少内存碎片和有效地重用键值状态来提高服务吞吐量[1]。正交系统工作通过 FlashAttention、FlashAttention-2、ORCA 和 DeepSpeed-Inference [2,3,4,5] 改进了注意力内核、连续批处理和分布式推理执行。这些技术主要是为数据中心服务而设计的，而我们的设置为端点提供了明确的权限，决定是否可以使用历史 KV 缓存、使用多少历史 KV 缓存，以及在每个生成步骤期间公开多少信息。

2.2 拆分和边云推理

分割计算在端点和云之间移动神经网络的不同部分。早期的协作智能系统，例如 Neurosurgeon，研究了移动设备和云服务器之间的层放置 [6]。拆分计算调查和层分区系统进一步表征了延迟、带宽和异构设备的权衡[7,8,9]。最近面向大语言模型的边缘云方法研究混合小/大模型、协作边缘推理和分割推测解码[10,11,12,13,14,15]。对于 LLM 来说，大型解码器堆栈、词汇投影和 KV 缓存引入了新的瓶颈。因此，我们的框架不仅分割模型层，还分割缓存访问、词汇投影、推测解码和特定于设备的传输逻辑，并将隐私暴露作为一流的设计目标。

2.3 推测解码

推测性解码通过使用较小的草稿模型来提出未来的令牌，然后由较大的目标模型进行验证，从而加速自回归生成[16, 17]。后来的工作通过基于树的验证、硬件感知草稿树、多个解码头、前瞻解码、知识蒸馏草稿模型和自推测解码扩展了这一想法[18,19,20,21,22,23,24,25]。在所提出的协作框架中，端点托管一个轻量级推测模块。云验证草稿令牌并更新 KV 缓存，允许接受的令牌减少交互轮次，而无需暴露完整的端点侧模块参数。

2.4 Parameter-Efficient Adaptation and Privacy

LoRA introduces trainable low-rank matrices into pretrained models and enables parameter-efficient adaptation while keeping backbone weights frozen [26]. Quantized LoRA, split LoRA, and privacy-oriented LoRA variants demonstrate that low-rank adaptation can be combined with compression, split learning, and privacy-aware training protocols [27, 28, 29, 30, 31, 32]. Distributed and locally protected LLM inference has also been studied using alternative privacy mechanisms such as local differential privacy and distributed secure inference [33, 34]. We use LoRA-adapted lightweight modules on the endpoint. Their weights remain local, which increases the difficulty of reconstructing transferred hidden states through model-parameter enumeration. This is not a formal differential-privacy guarantee, but it provides a practical system-level protection layer when combined with encrypted transport and restricted cache authorization.

3 System Overview

Figure 1 shows the proposed architecture. The endpoint and cloud communicate through encrypted and quantized tensor messages. The endpoint is responsible for user-facing preprocessing, language identification, embedding computation, cache-control parameter generation, local partial inference, speculative decoding, and final token sampling. The cloud is responsible for authorized decoder inference, KV-cache maintenance, speculative-token verification, and high-dimensional partial logits computation.

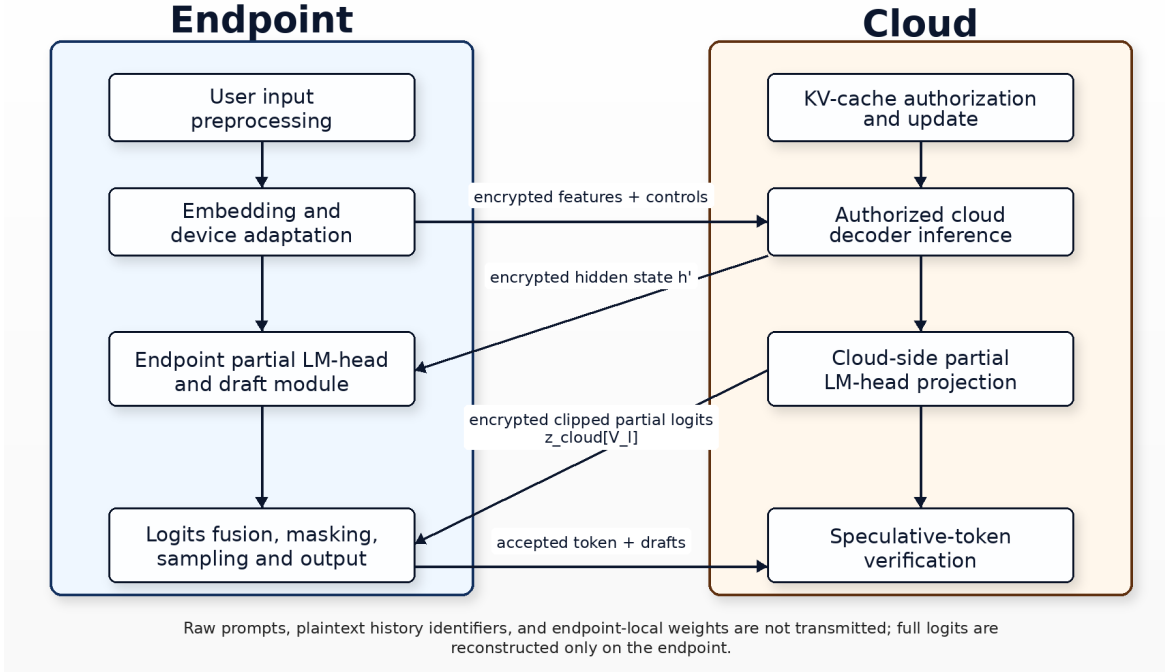


Figure 1: Overview of the edge-cloud collaborative LLM inference framework. The endpoint keeps privacy-sensitive control and lightweight computation local, while the cloud performs authorized decoder inference and high-dimensional projection.

The framework operates under three design principles. First, the endpoint controls whether the cloud may use historical KV-cache entries and how many entries are authorized. Second, raw user inputs and complete logits distributions are not transmitted during generation. Third, device-specific execution paths are selected according to endpoint capability: block-wise streaming for pure CPU endpoints, model slicing for GPU endpoints, and ONNX Runtime for resource-constrained edge devices.

2.4 参数高效的适配和隐私

LoRA 将可训练的低秩矩阵引入到预训练模型中，并在保持骨干权重冻结的同时实现参数高效的自适应[26]。量化 LoRA、分割 LoRA 和面向隐私的 LoRA 变体表明，低秩自适应可以与压缩、分割学习和隐私感知训练协议相结合 [27,28,29,30,31,32]。还使用本地差分隐私和分布式安全推理等替代隐私机制研究了分布式和本地保护的 LLM 推理 [33, 34]。我们在端点上使用适合 LoRA 的轻量级模块。它们的权重仍然是局部的，这增加了通过模型参数枚举重建转移隐藏状态的难度。这不是正式的差异隐私保证，但与加密传输和受限缓存授权相结合时，它提供了实用的系统级保护层。

3 系统概述

图 1 显示了建议的架构。端点和云通过加密和量化的张量消息进行通信。端点负责面向用户的预处理、语言识别、嵌入计算、缓存控制参数生成、本地部分推理、推测解码和最终令牌采样。云负责授权解码器推理、KV 缓存维护、推测令牌验证和高维部分逻辑计算。

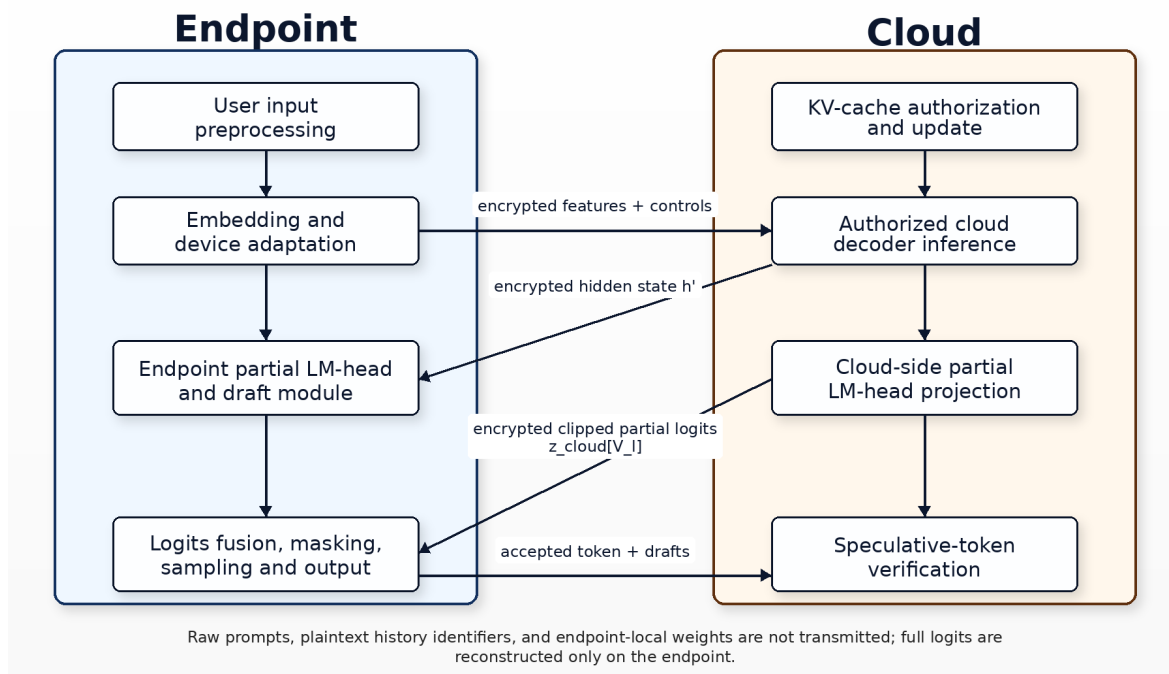


图 1：边缘-云协作 LLM 推理框架概述。端点将隐私敏感的控制和轻量级计算保留在本地，而云则执行授权的解码器推理和高维投影。

该框架根据三个设计原则运行。首先，端点控制云是否可以使用历史 KV 缓存条目以及授权多少条目。其次，原始用户输入和完整的 logits 分布在生成过程中不会传输。第三，根据端点能力选择特定于设备的执行路径：用于纯 CPU 端点的分块流、用于 GPU 端点的模型切片以及用于资源受限边缘设备的 ONNX 运行时。

4 Threat Model and Design Goals

We target deployment scenarios in which the endpoint is controlled by the user or by a trusted local application, while the cloud is operated by a service provider that follows the protocol but may be curious about user content. The cloud is therefore allowed to observe decrypted feature tensors that are required for authorized inference, cloud-side partial logits, token-level verification metadata, and cache-control parameters. It should not receive raw prompts, plaintext conversation-history identifiers, endpoint-local LoRA weights, or complete endpoint-side lightweight-module parameters. Network attackers may observe, replay, or modify packets, but they do not know the authenticated-encryption keys.

The design is not a substitute for secure multi-party computation, homomorphic encryption, trusted execution, or formal differential privacy. Instead, it pursues four system-level goals. First, the endpoint should explicitly authorize whether historical KV-cache states may be used and how many entries are exposed to a generation step. Second, communication should avoid transmitting raw text and full vocabulary distributions whenever a language-specific subset is sufficient. Third, endpoint-local lightweight modules should reduce the information available to an adversary who only observes cloud-side tensors. Fourth, the design should preserve practical deployability on CPU-only clients, GPU-equipped endpoints, and embedded edge devices.

5 Method

5.1 Notation

Let x denote a user input sequence and $T(\cdot)$ denote tokenization. The endpoint maps x to token indices $u = T(x)$ and then computes an initial hidden representation

$$h_0 = E_{\text{tok}}(u) + E_{\text{pos}}(p) + E_{\text{type}}(t), \quad (1)$$

where E_{tok} , E_{pos} , and E_{type} are token, positional, and type embeddings, respectively. The hidden dimension is denoted by D and the vocabulary size by V .

The endpoint also creates an inference-control tuple

$$c = (\text{flag}_{\text{Cache}}, L_{\text{Cache}}, d_{\text{type}}, \ell), \quad (2)$$

where $\text{flag}_{\text{Cache}}$ indicates whether historical KV-cache entries may be used, L_{Cache} is the authorized cache length, d_{type} describes the endpoint type, and ℓ is the language identifier. The identifier ℓ may be estimated from the prompt or set by user preference, for example Chinese-only, English-only, or mixed-language generation.

5.2 Device-Adaptive Feature Preparation

The endpoint adapts the initial feature tensor according to hardware capability. The design is compatible with common model-compression techniques for edge deployment, including post-training quantization, activation-aware quantization, pruning, distillation, and CPU-oriented inference optimization [35, 36, 37, 38, 39, 40].

Pure CPU endpoints. For CPU-only devices, the feature tensor is partitioned by columns into N_b blocks:

$$h_0 = \text{concat}(h_0^{(1)}, h_0^{(2)}, \dots, h_0^{(N_b)}). \quad (3)$$

Each block is transmitted with an order identifier and integrity metadata. This enables streaming: the endpoint can decrypt and process block k while receiving block $k + 1$, hiding part of the network latency behind local computation.

GPU-equipped endpoints. For endpoints with GPU acceleration, the local modules may be sliced by Transformer layer or attention head. A lightweight serving engine can use tensor parallelism and multi-session batching to accelerate embedding, speculative decoding, and local projection.

Edge devices. For embedded or IoT-class devices, endpoint modules are exported to ONNX and executed through ONNX Runtime. Quantization can be selected according to the available memory budget. The ONNX execution path also allows hardware-specific execution providers, such as vector instructions or neural-processing accelerators, to be used when available.

4 威胁模型和设计目标

我们的目标部署场景是，端点由用户或受信任的本地应用程序控制，而云由遵循协议但可能对用户内容感到好奇的服务提供商操作。因此，云可以观察授权推理、云端部分逻辑、令牌级验证元数据和缓存控制参数所需的解密特征张量。它不应接收原始提示、纯文本对话历史标识符、端点本地 LoRA 权重或完整的端点侧轻量级模块参数。网络攻击者可以观察、重放或修改数据包，但他们不知道经过身份验证的加密密钥。

该设计不能替代安全多方计算、同态加密、可信执行或形式差分隐私。相反，它追求四个系统级目标。首先，端点应明确授权是否可以使用历史 KV 缓存状态以及向生成步骤公开多少条目。其次，只要特定语言的子集足够，通信就应避免传输原始文本和完整的词汇分布。第三，端点本地轻量级模块应该减少仅观察云端张量的对手可用的信息。第四，设计应保留在仅使用 CPU 的客户端、配备 GPU 的端点和嵌入式边缘设备上的实际可部署性。

5 方法

5.1 符号

令 x 表示用户输入序列， $T(\cdot)$ 表示标记化。端点将 x 映射到标记索引 $u = T(x)$ ，然后计算初始隐藏表示

$$h_0 = E_{\text{tok}}(u) + E_{\text{pos}}(p) + E_{\text{type}}(t), \quad (1)$$

其中 E_{tok} 、 E_{pos} 和 E_{type} 分别是标记、位置和类型嵌入。隐藏维度由 D 表示，词汇量由 V 表示。

端点还创建一个推理控制元组

$$c = (\text{flag}_{\text{Cache}}, L_{\text{Cache}}, d_{\text{type}}, \ell), \quad (2)$$

其中 $\text{flag}_{\text{Cache}}$ 表示是否可以使用历史 KV 缓存条目， L_{Cache} 是授权的缓存长度， d_{type} 描述端点类型， ℓ 是语言标识符。标识符 ℓ 可以根据提示估计或根据用户偏好设置，例如纯中文、纯英文或混合语言生成。

5.2 设备自适应特征准备

端点根据硬件能力调整初始特征张量。该设计与边缘部署的常见模型压缩技术兼容，包括训练后量化、激活感知量化、修剪、蒸馏和面向 CPU 的推理优化 [35、36、37、38、39、40]。

纯 CPU 端点。对于仅使用 CPU 的设备，特征张量按列分区 mms 进入 N_b 块：

$$h_0 = \text{concat}(h_0^{(1)}, h_0^{(2)}, \dots, h_0^{(N_b)}). \quad (3)$$

每个块都与顺序标识符和完整性元数据一起传输。这使得流式传输成为可能：端点可以在接收块 $k+1$ 的同时解密和处理块 k ，从而将部分网络延迟隐藏在本地计算后面。

配备 GPU 的端点。对于具有 GPU 加速的端点，本地模块可以通过 Transformer 层或注意力头进行切片。轻量级服务引擎可以使用张量并行性和多会话批处理来加速嵌入、推测解码和本地投影。

边缘设备。对于嵌入式或物联网级设备，端点模块导出到 ONNX 并通过 ONNX 运行时执行。可以根据可用的内存预算来选择量化。ONNX 执行路径还允许在可用时使用特定于硬件的执行提供程序，例如向量指令或神经处理加速器。

5.3 Authorized Cloud Decoder Inference

The cloud receives the adapted feature tensor and the endpoint-generated control tuple. If cache usage is authorized, the cloud selects at most L_{Cache} historical KV entries for the current session. Otherwise, it ignores existing cache entries and performs decoding from the transmitted feature tensor. This endpoint-controlled cache interface is complementary to KV-cache compression and streaming-cache management methods [41, 42].

For a decoder layer, the cloud computes

$$Q = hW_Q, \quad (4)$$

$$K = \text{concat}(K_{\text{Cache}}^{1:L_{\text{Cache}}}, hW_K), \quad (5)$$

$$V = \text{concat}(V_{\text{Cache}}^{1:L_{\text{Cache}}}, hW_V), \quad (6)$$

$$a = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \quad (7)$$

$$h' = \text{FFN}(a), \quad (8)$$

where W_Q , W_K , and W_V are projection matrices, M is the autoregressive attention mask, and h' is the cloud-produced hidden state before the final vocabulary projection.

For CPU endpoints, the returned hidden state can be partitioned according to the same block rule and streamed back. For GPU and edge endpoints, the cloud returns the complete hidden state. All returned tensors are quantized and encrypted before transmission.

5.4 Distributed Vocabulary Projection

The final language-model head is expensive because it maps each hidden vector to a distribution over a large vocabulary. Let the full output projection be

$$z = h'W_{\text{lm}} + b, \quad (9)$$

$$W_{\text{lm}} \in \mathbb{R}^{D \times V}, \quad b \in \mathbb{R}^V.$$

The framework splits both the hidden vector and the corresponding rows of the LM-head weight matrix along the hidden dimension:

$$h' = \text{concat}(h_{\text{end}}, h_{\text{cloud}}), \quad D_{\text{end}} + D_{\text{cloud}} = D, \quad (10)$$

$$W_{\text{lm}} = \begin{bmatrix} W_{\text{end}} \\ W_{\text{cloud}} \end{bmatrix},$$

where $W_{\text{end}} \in \mathbb{R}^{D_{\text{end}} \times V}$ and $W_{\text{cloud}} \in \mathbb{R}^{D_{\text{cloud}} \times V}$. The endpoint computes

$$z_{\text{end}} = h_{\text{end}}W_{\text{end}}, \quad (11)$$

while the cloud computes

$$z_{\text{cloud}} = h_{\text{cloud}}W_{\text{cloud}} + b. \quad (12)$$

The endpoint then reconstructs the logits by element-wise addition:

$$z = z_{\text{end}} + z_{\text{cloud}}. \quad (13)$$

For language-restricted generation, the cloud does not return all V dimensions. The endpoint determines a vocabulary subset $\mathcal{V}_\ell$ from the language identifier ℓ and sends only the subset identifier or a compact subset mask to the cloud. The cloud returns $z_{\text{cloud}}[\mathcal{V}_\ell]$ after clipping, quantization, and encryption. The endpoint computes the matching local component $z_{\text{end}}[\mathcal{V}_\ell]$, fuses the two partial logits, and assigns $-\infty$ to all dimensions outside $\mathcal{V}_\ell$:

$$\tilde{z}_i = \begin{cases} z_{\text{end},i} + z_{\text{cloud},i}, & i \in \mathcal{V}_\ell, \\ -\infty, & i \notin \mathcal{V}_\ell. \end{cases} \quad (14)$$

The next token is sampled from

$$p(y_t \mid y_{<t}, x) = \text{softmax}(\tilde{z}/\tau), \quad (15)$$

where τ is the sampling temperature. In mixed-language mode, $\mathcal{V}_\ell$ is set to the complete vocabulary or to the union of selected language subsets. This formulation makes explicit that both partial projections have vocabulary-dimensional outputs and that the bias term is applied exactly once.

5.3 授权云解码器推理

云接收调整后的特征张量和端点生成的控制元组。如果授权使用缓存，则云最多为当前会话选择 L_{Cache} 条历史 KV 条目。否则，它会忽略现有的缓存条目并根据传输的特征张量执行解码。这种端点控制的缓存接口是对 KV 缓存压缩和流缓存管理方法的补充 [41, 42]。

对于解码器层，云计算

$$Q = hW_Q, \quad (4)$$

$$K = \text{concat}(K_{\text{Cache}}^{1:L_{\text{Cache}}}, hW_K), \quad (5)$$

$$V = \text{concat}(V_{\text{Cache}}^{1:L_{\text{Cache}}}, hW_V), \quad (6)$$

$$a = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \quad (7)$$

$$h' = \text{FFN}(a), \quad (8)$$

其中 W_Q 、 W_K 和 W_V 是投影矩阵， M 是自回归注意力掩模， h' 是最终词汇投影之前云生成的隐藏状态。

对于 CPU 端点，返回的隐藏状态可以根据相同的块规则进行分区并流回。对于 GPU 和边缘端点，云返回完整的隐藏状态。所有返回的张量在传输前都经过量化和加密。

5.4 分布式词汇投影

最终的语言模型头很昂贵，因为它将每个隐藏向量映射到大词汇量的分布。令完整输出投影为

$$z = h'W_{\text{lm}} + b, \quad (9)$$

$$W_{\text{lm}} \in \mathbb{R}^{D \times V}, \quad b \in \mathbb{R}^V.$$

该框架沿隐藏维度分割隐藏向量和 LM-head 权重矩阵的相应行：

$$h' = \text{concat}(h_{\text{end}}, h_{\text{cloud}}), \quad D_{\text{end}} + D_{\text{cloud}} = D, \quad (10)$$

$$W_{\text{lm}} = \begin{bmatrix} W_{\text{end}} \\ W_{\text{cloud}} \end{bmatrix},$$

其中 $W_{\text{end}} \in \mathbb{R}^{D_{\text{end}} \times V}$ 和 $W_{\text{cloud}} \in \mathbb{R}^{D_{\text{cloud}} \times V}$ 。端点计算

$$z_{\text{end}} = h_{\text{end}}W_{\text{end}}, \quad (11)$$

当云计算时

$$z_{\text{cloud}} = h_{\text{cloud}}W_{\text{cloud}} + b. \quad (12)$$

然后端点通过逐元素加法重建 logits：

$$z = z_{\text{end}} + z_{\text{cloud}}. \quad (13)$$

对于语言限制的生成，云不会返回所有 V 维度。端点根据语言标识符 ℓ 确定词汇子集 $\mathcal{V}_\ell$ ，并仅将子集标识符或紧凑子集掩码发送到云。云端经过裁剪、量化和加密后返回 $z_{\text{cloud}}[\mathcal{V}_\ell]$ 。端点计算匹配的本地分量 $z_{\text{end}}[\mathcal{V}_\ell]$ ，融合两个部分 logits，并将 $-\infty$ 分配给 $\mathcal{V}_\ell$ 之外的所有维度：

$$\tilde{z}_i = \begin{cases} z_{\text{end},i} + z_{\text{cloud},i}, & i \in \mathcal{V}_\ell, \\ -\infty, & i \notin \mathcal{V}_\ell. \end{cases} \quad (14)$$

下一个令牌是从采样的

$$p(y_t | y_{<t}, x) = \text{softmax}(\tilde{z}/\tau), \quad (15)$$

其中 τ 是采样温度。在混合语言模式下， $\mathcal{V}_\ell$ 设置为完整词汇表或所选语言子集的并集。该公式明确表明，两个部分投影都具有词汇维度输出，并且偏差项仅应用一次。

5.5 Endpoint Speculative Decoding and Cloud Verification

The endpoint contains a lightweight draft module g_ϕ implemented as either an MLP or a small Transformer block followed by an MLP. Given the cloud-returned hidden state, the module proposes N future tokens:

$$s_t = g_\phi(h'_t), \quad (16)$$

$$\hat{y}_{t:t+N-1} = \arg \max \text{Proj}(s_t). \quad (17)$$

The draft length N is device-dependent: CPU and edge endpoints use shorter drafts to control local compute, while GPU-equipped endpoints use longer drafts to reduce interaction rounds.

For each generation step, the endpoint sends the sampled incremental token and the proposed draft sequence to the cloud. The cloud verifies the draft tokens with the target decoder under the endpoint-authorized KV-cache window. Accepted draft tokens are committed to the cloud-side KV cache and reused in subsequent steps; rejected draft tokens trigger recomputation from the last accepted position. This verification keeps the cloud KV cache consistent with the authoritative target model. Because quantization, language masking, and endpoint-side approximation can slightly change the sampling distribution, the workflow should be regarded as approximately target-preserving rather than strictly distribution-preserving.

Figure 2 summarizes the generation loop in a compact form.

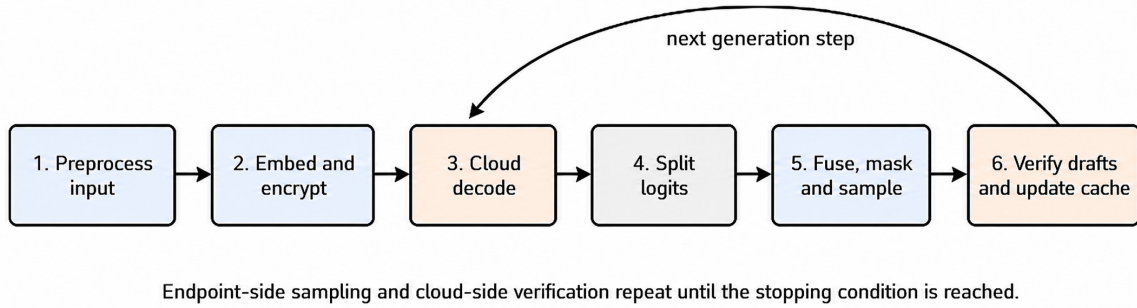


Figure 2: Autoregressive generation loop with endpoint-controlled KV-cache usage, split logits computation, language-adaptive masking, and cloud-side speculative-token verification.

Algorithm 1 Endpoint-authorized collaborative decoding

Require: user input x , endpoint type d_{type} , maximum response length T

- 1: Endpoint tokenizes x , computes h_0 , detects language identifier ℓ , and sets $(\text{flag}_{\text{Cache}}, L_{\text{Cache}})$.
 - 2: Endpoint prepares h_0 according to d_{type} and sends encrypted tensor blocks plus the control tuple to the cloud.
 - 3: Cloud performs authorized decoder inference and returns encrypted hidden states or hidden-state blocks.
 - 4: **for** $t = 1$ to T **do**
 - 5: Endpoint computes $z_{\text{end}}[\mathcal{V}_\ell]$ and draft tokens $\hat{y}_{t:t+N-1}$.
 - 6: Cloud computes and returns $z_{\text{cloud}}[\mathcal{V}_\ell]$ under the authorized cache window.
 - 7: Endpoint fuses partial logits, applies language masking, and samples y_t .
 - 8: Endpoint sends y_t and $\hat{y}_{t:t+N-1}$ to the cloud for verification.
 - 9: Cloud commits accepted tokens, recomputes from the first rejected token if necessary, and updates the KV cache.
 - 10: **if** an end-of-sequence token or stopping rule is reached **then**
 - 11: **break**
 - 12: **end if**
 - 13: **end for**
 - 14: Endpoint decodes the generated token sequence and returns the final response.
-

5.5 端点推测解码和云端验证

端点包含一个轻量级草稿模块 g_ϕ ，其实现为 MLP 或后跟 MLP 的小型 Transformer 块。给定云返回的隐藏状态，该模块提出 N 未来令牌：

$$s_t = g_\phi(h'_t), \quad (16)$$

$$\hat{y}_{t:t+N-1} = \arg \max \text{Proj}(s_t). \quad (17)$$

草稿长度 N 取决于设备：CPU 和边缘端点使用较短的草稿来控制本地计算，而配备 GPU 的端点使用较长的草稿来减少交互轮次。

对于每个生成步骤，端点将采样的增量令牌和建议的草稿序列发送到云。云在端点授权的 KV 缓存窗口下使用目标解码器验证草稿令牌。接受的草稿令牌将提交到云端 KV 缓存并在后续步骤中重复使用；被拒绝的草稿令牌会触发从最后接受的位置重新计算。该验证使云端 KV 缓存与权威目标模型保持一致。由于量化、语言屏蔽和端点侧近似可以稍微改变采样分布，因此工作流程应被视为近似目标保留而不是严格分布保留。

图 2 以紧凑的形式总结了生成循环。

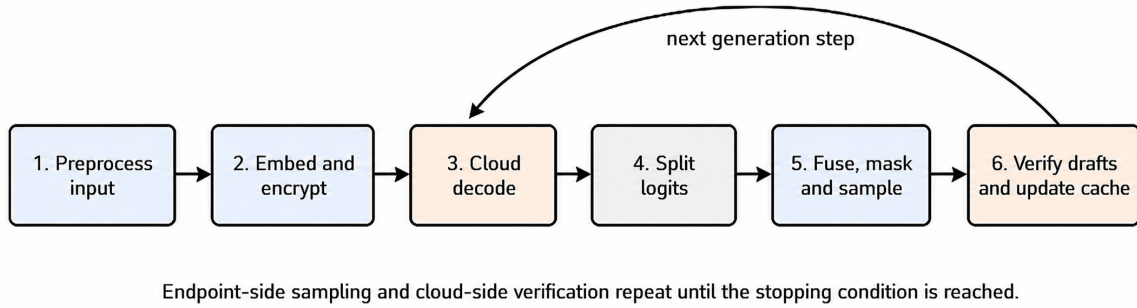


图 2：具有端点控制的 KV 缓存使用、分割 logits 计算、语言自适应屏蔽和云端推测令牌验证的自回归生成循环。

算法1 端点授权协同解码

要求：用户输入 x 、端点类型 d_{type} 、最大响应长度 T 1: 端点标记化 x 、计算 h_0 、检测语言标识符 ℓ 并设置（标志 Cache , L_{Cache} ）。 2: 端点根据 d_{type} 准备 h_0 ，并将加密的张量块加上控制元组发送到云端。 3: 云执行授权解码器推理并返回加密的隐藏状态或隐藏状态块。 4: 对于 $t = 1$ 到 T 执行 5: 端点计算 $z_{\text{end}}[\mathcal{V}_\ell]$ 和草案令牌 $\hat{y}_{t:t+N-1}$ 。 6: 云端在授权缓存窗口下计算并返回 $z_{\text{cloud}}[\mathcal{V}_\ell]$ 。 7: 端点融合部分逻辑，应用语言屏蔽，并采样 y_t 。 8: 端点将 y_t 和 $\hat{y}_{t:t+N-1}$ 发送到云端进行验证。 9: 云提交接受的令牌，如有必要，从第一个拒绝的令牌重新计算，并更新 KV 缓存。 10: 如果达到序列结束令牌或停止规则，则 11: 中断 12: 结束 if 13: 结束 for 14: 端点对生成的令牌序列进行解码并返回最终响应。

5.6 Transport Protection and Hidden-State Protection

The system combines network-level authenticated encryption with endpoint-local model-state protection. The scope of each layer follows the threat model in Section 4.

Encrypted tensor transmission. Before transmission, tensor elements are quantized to a configurable bit width b :

$$q = Q_b(r), \quad (18)$$

where r denotes a real-valued tensor element or vector. The quantized payload is encrypted with AES-GCM [43]:

$$C = \text{AES-GCM}_{K,IV}(q, \text{aad}), \quad (19)$$

where K is a pre-shared or derived key, IV is a nonce, and aad contains authenticated session, block, and cache-control metadata. CPU-block mode assigns independent nonces to different tensor blocks, while GPU and edge modes assign nonces by session and message type. AES-GCM protects payloads against network-level eavesdropping and tampering; it does not prevent the authorized cloud from observing decrypted tensors that are required for cloud-side inference. Compared with cryptographic transformer inference based on secure multi-party computation, homomorphic encryption, or trusted execution environments [44, 45, 46, 47], our design uses conventional authenticated encryption for transport and treats endpoint-local modules as a practical exposure-reduction mechanism rather than as a formal secure-computation protocol.

Endpoint-local LoRA modules. The endpoint lightweight modules are adapted with LoRA [26]. For a base weight W_0 , LoRA introduces a low-rank update

$$W = W_0 + \Delta W, \quad \Delta W = BA, \quad (20)$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are trainable low-rank matrices. During adaptation, the base model is frozen and only the LoRA parameters are updated. The adapted lightweight weights, draft-module parameters, and local projection weights remain on the endpoint and are not transmitted to the cloud. This limits the model information available to a cloud-side inversion attempt, although it is not a formal cryptographic or differential-privacy guarantee.

KV-cache authorization. The endpoint controls $\text{flag}_{\text{Cache}}$ and L_{Cache} . A new session can disable cache reuse, while an ongoing session can reduce the authorized cache window when privacy requirements are high. The cloud maintains physical KV-cache entries for efficiency, but each generation request includes an endpoint-generated authorization tuple that determines whether those entries may be used.

6 Implementation Considerations

6.1 Device Profiles

Table 1 summarizes the main execution modes.

Table 1: Device-adaptive execution profiles.

Endpoint type	Local execution strategy	Communication strategy
Pure CPU	Block-wise embedding, lightweight MLP, local draft module, SIMD-friendly matrix multiplication	Stream hidden-state blocks, overlap decryption and computation, use per-block integrity metadata
GPU-equipped	Model slicing, tensor parallelism, local batching, multi-session scheduling	Transmit complete hidden states, use session-level encryption and batched requests
Edge device	Quantized ONNX Runtime execution [48], hardware execution providers, compact draft module	Use quantized tensors, avoid redundant format conversion, support low-memory execution

6.2 Language-Adaptive Logits Compression

Language-adaptive masking reduces the size of returned logits when generation is restricted to a target language. In a Chinese-only or English-only mode, only the corresponding vocabulary subset must be transmitted and fused. In mixed-language mode, the full vocabulary or a union of language subsets can be retained. The subset mapping can be precomputed from tokenizer metadata and updated when the model vocabulary changes.

5.6 传输保护和隐藏状态保护

该系统将网络级身份验证加密与端点本地模型状态保护相结合。每层的范围遵循第 4 节中的威胁模型。

加密张量传输。在传输之前，张量元素被量化为配置可用位宽 b ：

$$q = Q_b(r), \quad (18)$$

其中 r 表示实值张量元素或向量。量化的有效负载使用 AES-GCM [43] 加密：

$$C = \text{AES-GCM}_{K,IV}(q, \text{aad}), \quad (19)$$

其中 K 是预共享或派生密钥， IV 是随机数， aad 包含经过身份验证的会话、块和缓存控制元数据。CPU 块模式将独立的随机数分配给不同的张量块，而 GPU 和边缘模式则根据会话和消息类型分配随机数。AES-GCM 保护有效负载免遭网络级窃听和篡改；它不会阻止授权云观察云端推理所需的解密张量。与基于安全多方计算、同态加密或可信执行环境的密码变换器推理[44,45,46,47]相比，我们的设计使用传统的认证加密进行传输，并将端点本地模块视为一种实用的暴露减少机制，而不是一种正式的安全计算协议。

端点本地 LoRA 模块。端点轻量级模块采用 LoRA [26]。对于基本权重 W_0 ，LoRA 引入了低秩更新

$$W = W_0 + \Delta W, \quad \Delta W = BA, \quad (20)$$

其中 $A \in \mathbb{R}^{r \times d}$ 和 $B \in \mathbb{R}^{d \times r}$ 是可训练的低秩矩阵。在适应过程中，基础模型被冻结，仅更新 LoRA 参数。调整后的轻量级权重、草稿模块参数和本地投影权重保留在端点上，不会传输到云端。这限制了云端反演尝试可用的模型信息，尽管它不是正式的加密或差异隐私保证。

KV-缓存授权。端点控制标志 Cache 和 L_{Cache} 。新会话可以禁用缓存重用，而当隐私要求较高时，正在进行的会话可以减少授权缓存窗口。云维护物理 KV 缓存条目以提高效率，但每个生成请求都包含端点生成的授权元组，用于确定是否可以这些条目。

6 实施注意事项

6.1 设备配置文件

表 1 总结了主要的执行模式。

表 1: 设备自适应执行配置文件。

Endpoint type	Local execution strategy	Communication strategy
Pure CPU	Block-wise embedding, lightweight MLP, local draft module, SIMD-friendly matrix multiplication	Stream hidden-state blocks, overlap decryption and computation, use per-block integrity metadata
GPU-equipped	Model slicing, tensor parallelism, local batching, multi-session scheduling	Transmit complete hidden states, use session-level encryption and batched requests
Edge device	Quantized ONNX Runtime execution [48], hardware execution providers, compact draft module	Use quantized tensors, avoid redundant format conversion, support low-memory execution

6.2 语言自适应 Logits 压缩

当生成仅限于目标语言时，语言自适应屏蔽会减少返回的 logits 的大小。在纯中文或纯英文模式下，只需要传输和融合相应的词汇子集。在混合语言模式下，可以保留完整词汇或语言子集的联合。子集映射可以根据分词器元数据预先计算，并在模型词汇表更改时更新。

6.3 Failure Handling

If cache authorization fails or the cloud does not find a valid session cache, the system falls back to cache-free decoding. If a CPU block is lost or fails integrity verification, the cloud retransmits that block. If speculative verification fails, the cloud recomputes from the last accepted token and updates the cache with the corrected state.

7 Experimental Evaluation

This section evaluates the proposed edge-cloud collaborative inference framework against conventional serving strategies. We implemented a prototype with a PyTorch cloud decoder, an endpoint runtime for local embedding/projection/draft modules, QUIC-based message transport, 8-bit tensor quantization, and AES-GCM authenticated encryption. The evaluation includes latency, throughput, communication volume, component ablations, sensitivity analyses, generation quality, and privacy-exposure accounting.

7.1 Research Questions

The evaluation is organized around four questions:

- **RQ1:** Does the proposed collaborative workflow reduce end-to-end generation latency compared with cloud-only and conventional split-inference baselines?
- **RQ2:** How much communication volume is saved by split vocabulary projection and language-adaptive logits compression?
- **RQ3:** Which components contribute most to latency, throughput, and bandwidth improvements?
- **RQ4:** What privacy-relevant information remains visible under different serving strategies?

7.2 Baselines

We compare the proposed framework with four baselines.

- **Cloud-only:** the endpoint sends the raw prompt and a conversation identifier to the cloud; the cloud performs all prefill and decoding and returns text tokens.
- **Cloud-only encrypted transport:** the same workflow as cloud-only, but the request and response payloads are protected in transit. This baseline protects the network channel but does not reduce cloud-side visibility.
- **Naive split inference:** the endpoint computes embeddings and the cloud returns hidden states or full logits without endpoint-controlled cache authorization, language-adaptive logits clipping, or endpoint-side speculative decoding.
- **Endpoint-only lightweight:** a compact local model performs generation without cloud assistance. This baseline is feasible only for the GPU and edge profiles and illustrates the quality-efficiency trade-off of pure local inference.

7.3 Experimental Setup

We instantiate the target model as a 7B-class decoder-only Transformer with hidden size $D = 4096$ and vocabulary size $V = 150,000$, matching the split dimensions used by the collaborative design. The endpoint processes $D_{\text{end}} = 96$ hidden dimensions, and the cloud processes $D_{\text{cloud}} = 4000$ dimensions. Unless otherwise stated, prompts are truncated or padded to 512 input tokens and responses are generated for 128 tokens. Tensor payloads and clipped logits are quantized to 8 bits before AES-GCM encryption. Chinese-only generation keeps 40% of the vocabulary, English-only generation keeps 30%, and mixed-language generation keeps the full vocabulary.

Table 2 summarizes the deployment profiles. The cloud server hosts the full decoder stack, maintains KV-cache entries, verifies speculative tokens, and computes the cloud-side LM-head projection. The CPU endpoint uses $N_b = 4$ streamed blocks and a rank-4 LoRA local module. The GPU endpoint uses a rank-8 local module, local batching, and speculative length $N = 5$. The edge endpoint uses an 8-bit ONNX Runtime module and speculative length $N = 2$.

6.3 故障处理

如果缓存授权失败或云找不到有效的会话缓存，系统将回退到无缓存解码。如果 CPU 块丢失或完整性验证失败，云会重新传输该块。如果推测性验证失败，云会根据最后接受的令牌重新计算，并使用更正后的状态更新缓存。

7 实验评价

本节根据传统服务策略评估所提出的边缘-云协作推理框架。我们使用 PyTorch 云解码器、用于本地嵌入/投影/草稿模块的端点运行时、基于 QUIC 的消息传输、8 位张量量化和 AES-GCM 验证加密来实现原型。评估包括延迟、吞吐量、通信量、组件消融、敏感性分析、生成质量和隐私暴露核算。

7.1 研究问题

评估围绕四个问题进行：

- RQ1：与纯云和传统的分割推理基线相比，所提出的协作工作流程是否减少了端到端生成延迟？
- RQ2：分割词汇投影和语言自适应 Logits 压缩可以节省多少通信量？
- RQ3：哪些组件对延迟、吞吐量和带宽改进贡献最大？
- RQ4：在不同的服务策略下，哪些隐私相关信息仍然可见？

7.2 基线

我们将提议的框架与四个基线进行比较。

- 仅云：端点将原始提示和对话标识符发送到云；云执行所有预填充和解码并返回文本标记。
- 仅云加密传输：与仅云相同的工作流程，但请求和响应有效负载在传输过程中受到保护。该基线可以保护网络通道，但不会降低云端可见性。
- 朴素分割推理：端点计算嵌入，云返回隐藏状态或完整 logits，无需端点控制的缓存授权、语言自适应 logits 裁剪或端点端推测解码。
- 仅端点轻量级：紧凑的本地模型无需云辅助即可执行生成。该基线仅适用于 GPU 和边缘配置文件，并说明了纯本地推理的质量与效率权衡。

7.3 实验设置

我们将目标模型实例化为 7B 级仅解码器 Transformer，其隐藏大小为 $D = 4096$ ，词汇大小为 $V = 150,000$ ，与协作设计使用的分割维度相匹配。端点处理 $D_{\text{end}} = 96$ 个隐藏维度，云处理 $D_{\text{cloud}} = 4000$ 个维度。除非另有说明，否则提示将被截断或填充为 512 个输入标记，并为 128 个标记生成响应。在 AES-GCM 加密之前，张量有效负载和裁剪后的 logits 被量化为 8 位。纯中文一代保留 40% 的词汇量，纯英语一代保留 30%，混合语言一代保留完整词汇量。

表 2 总结了部署配置文件。云服务器托管完整的解码器堆栈、维护 KV 缓存条目、验证推测令牌并计算云端 LM-head 投影。CPU 端点使用 $N_b = 4$ 个流块和一个 4 级 LoRA 本地模块。GPU 端点使用 8 级本地模块、本地批处理和推测长度 $N = 5$ 。边缘端点使用 8 位 ONNX 运行时模块和推测长度 $N = 2$ 。

Table 2: Prototype hardware and runtime configuration.

Profile	Hardware	Runtime configuration
Cloud	NVIDIA H20 96GB GPU, 64 vCPU host, 512GB RAM	PyTorch decoder service, KV-cache manager, partial LM-head projection, QUIC endpoint
CPU endpoint	12-core x86 CPU, 32GB RAM, no discrete GPU	SIMD matrix kernels, four streamed tensor blocks, rank-4 LoRA module
GPU endpoint	Consumer GPU with 8GB VRAM, 32GB host RAM	Local batching, sliced lightweight modules, rank-8 LoRA module, draft length $N = 5$
Edge endpoint	Embedded ARM device with 8GB unified memory	ONNX Runtime, 8-bit quantized lightweight modules, draft length $N = 2$

All endpoint-cloud experiments use a controlled network with 20 ms round-trip latency, 2 ms jitter, and a 100 Mbps bandwidth limit. We evaluate 1,000 prompts: 400 Chinese prompts, 400 English prompts, and 200 mixed-language prompts. Each configuration is repeated five times after 100 warm-up prompts; tables report the mean and, where space permits, the standard deviation. The cloud service runs on Ubuntu 22.04 with CUDA 12.4 and PyTorch 2.4. Endpoint components use Python 3.10, ONNX Runtime 1.18 for the embedded profile, and a QUIC/HTTP3 transport layer implemented with user-space message framing. Cloud-side decoding uses bf16 weights, while transmitted tensors and clipped logits use INT8 payloads unless otherwise specified. Latency includes endpoint tokenization, embedding, quantization, AES-GCM encryption/decryption, network transfer, cloud decoding, partial projection, draft verification, logits fusion, sampling, and KV-cache update. Model loading and first-time runtime initialization are excluded.

7.4 Metrics

We report average decode latency in milliseconds per generated token, end-to-end latency for a 128-token response, accepted-token throughput, downlink payload per generated token, draft-token acceptance rate, task-level generation quality, and privacy-exposure indicators. Throughput is computed from committed output tokens per second. Communication volume includes encrypted tensor payloads, clipped logits, authentication tags, block metadata, and padding, but excludes fixed connection setup overhead.

7.5 End-to-End Latency

Table 3 reports the latency comparison. The proposed framework consistently outperforms cloud-only and naive split inference across all endpoint profiles. The GPU endpoint obtains the largest gain because local batching and endpoint-side draft verification reduce interaction rounds. The CPU endpoint also benefits from block-wise streaming, which overlaps downstream transfer with local decryption and partial projection.

Table 3: End-to-end decode latency across endpoint profiles. Lower is better.

Method	CPU endpoint ms/token	GPU endpoint ms/token	Edge endpoint ms/token
Cloud-only	245.3 $\pm$ 5.8	239.1 $\pm$ 5.1	253.2 $\pm$ 6.4
Cloud-only encrypted transport	257.4 $\pm$ 6.1	249.3 $\pm$ 5.6	265.1 $\pm$ 6.8
Naive split inference	214.5 $\pm$ 4.9	178.2 $\pm$ 4.4	224.0 $\pm$ 5.7
Endpoint-only lightweight	–	92.1 $\pm$ 3.0	185.3 $\pm$ 5.2
Proposed framework	151.4 $\pm$ 4.1	96.0 $\pm$ 2.8	164.0 $\pm$ 4.6

For a 128-token response, the proposed framework achieves average end-to-end latencies of 19.4 s on the CPU endpoint, 12.3 s on the GPU endpoint, and 21.0 s on the edge endpoint. Relative to naive split inference, the reductions are 29.4%, 46.1%, and 26.8%, respectively. Although the endpoint-only lightweight model is faster on the GPU profile, it uses a smaller local model and therefore does not preserve the target cloud model as reliably as the verified collaborative workflow.

7.6 Throughput and Bandwidth

Table 4 reports throughput and downlink communication volume under different language modes. The communication volume is lowest in English-only mode because only 30% of vocabulary dimensions are retained. In mixed-language

表 2: 原型硬件和运行时配置。

Profile	Hardware	Runtime configuration
Cloud	NVIDIA H20 96GB GPU, 64 vCPU host, 512GB RAM	PyTorch decoder service, KV-cache manager, partial LM-head projection, QUIC endpoint
CPU endpoint	12-core x86 CPU, 32GB RAM, no discrete GPU	SIMD matrix kernels, four streamed tensor blocks, rank-4 LoRA module
GPU endpoint	Consumer GPU with 8GB VRAM, 32GB host RAM	Local batching, sliced lightweight modules, rank-8 LoRA module, draft length $N = 5$
Edge endpoint	Embedded ARM device with 8GB unified memory	ONNX Runtime, 8-bit quantized lightweight modules, draft length $N = 2$

所有端点-云实验均使用具有 20 毫秒往返延迟、2 毫秒抖动和 100 Mbps 带宽限制的受控网络。我们评估了 1,000 个提示: 400 个中文提示、400 个英文提示和 200 个混合语言提示。每个配置在 100 次预热提示后重复五次; 表格报告平均值, 并且在空间允许的情况下报告标准差。该云服务在带有 CUDA 12.4 和 PyTorch 2.4 的 Ubuntu 22.04 上运行。端点组件使用 Python 3.10、ONNX Runtime 1.18 作为嵌入式配置文件, 以及通过用户空间消息帧实现的 QUIC/HTTP3 传输层。除非另有说明, 云端解码使用 bf16 权重, 而传输张量和剪切 logits 使用 INT8 有效负载。延迟包括端点标记化、嵌入、量化、AES-GCM 加密/解密、网络传输、云解码、部分投影、草稿验证、logits 融合、采样和 KV 缓存更新。不包括模型加载和首次运行时初始化。

7.4 指标

我们报告每个生成的令牌的平均解码延迟 (以毫秒为单位)、128 个令牌响应的端到端延迟、接受的令牌吞吐量、每个生成的令牌的下行链路负载、草稿令牌接受率、任务级生成质量和隐私暴露指标。吞吐量是根据每秒提交的输出令牌计算的。通信量包括加密的张量有效负载、剪切的逻辑、身份验证标签、块元数据和填充, 但不包括固定的连接设置开销。

7.5 端到端延迟

表 3 报告了延迟比较。所提出的框架在所有端点配置文件中始终优于仅云和天真的分割推理。GPU 端点获得最大收益, 因为本地批处理和端点侧草稿验证减少了交互次数。CPU 端点还受益于逐块流, 它将下游传输与本地解密和部分投影重叠。

表 3: 跨端点配置文件的端到端解码延迟。越低越好。

Method	CPU endpoint ms/token	GPU endpoint ms/token	Edge endpoint ms/token
Cloud-only	245.3 $\pm$ 5.8	239.1 $\pm$ 5.1	253.2 $\pm$ 6.4
Cloud-only encrypted transport	257.4 $\pm$ 6.1	249.3 $\pm$ 5.6	265.1 $\pm$ 6.8
Naive split inference	214.5 $\pm$ 4.9	178.2 $\pm$ 4.4	224.0 $\pm$ 5.7
Endpoint-only lightweight	–	92.1 $\pm$ 3.0	185.3 $\pm$ 5.2
Proposed framework	151.4 $\pm$ 4.1	96.0 $\pm$ 2.8	164.0 $\pm$ 4.6

对于 128 个令牌的响应, 所提出的框架在 CPU 端点上实现了 19.4 秒的平均端到端延迟, 在 GPU 端点上实现了 12.3 秒的平均端到端延迟, 在边缘端点上实现了 21.0 秒的平均端到端延迟。相对于朴素分割推理, 分别减少了 29.4%、46.1% 和 26.8%。尽管仅限端点的轻量级模型在 GPU 配置文件上速度更快, 但它使用较小的本地模型, 因此无法像经过验证的协作工作流程那样可靠地保留目标云模型。

7.6 吞吐量和带宽

表4报告了不同语言模式下的吞吐量和下行通信量。纯英语模式下的流量最低, 因为仅保留了 30% 的词汇维度。混合语言

mode, the full vocabulary is retained, so most of the logits-compression benefit disappears, but speculative decoding and split computation still improve latency.

Table 4: Throughput and per-token downlink payload under different language modes.

Endpoint	Language mode	Throughput tokens/s	Downlink KB/token	Compression vs. full logits
CPU	Chinese-only	6.6 ± 0.2	64.2	56.2%
CPU	English-only	6.9 ± 0.2	49.1	66.5%
CPU	Mixed	5.4 ± 0.1	154.0	0.0%
GPU	Chinese-only	10.4 ± 0.3	62.1	57.6%
GPU	English-only	11.5 ± 0.3	47.8	67.4%
GPU	Mixed	8.6 ± 0.2	151.3	0.0%
Edge	Chinese-only	6.1 ± 0.2	63.8	56.5%
Edge	English-only	6.5 ± 0.2	48.6	66.8%
Edge	Mixed	4.8 ± 0.1	153.1	0.0%

With 8-bit logits, a full 150,000-dimensional vector requires approximately 146.5 KB before metadata and authentication tags. The measured payload can be approximated as

$$\text{Payload} \approx |\mathcal{V}_\ell| \cdot b/8 + B_{\text{tag}} + B_{\text{meta}} + B_{\text{pad}}, \quad (21)$$

where B_{tag} , B_{meta} , and B_{pad} denote AES-GCM tags, block/session metadata, and padding. The measured values are therefore slightly larger than the theoretical clipped-logit sizes.

7.7 Ablation Study

Table 5 isolates the contribution of each component. Removing speculative decoding substantially increases latency because the cloud must perform more verified target-model decoding rounds. Removing language-adaptive compression mainly increases downlink payload. Removing split vocabulary projection increases both latency and payload because the endpoint must wait for larger cloud-side logits. Removing CPU block overlap hurts CPU latency, while removing endpoint KV-cache control has little latency impact but weakens the privacy-control objective.

Table 5: Ablation results for major framework components. Lower latency and payload are better.

Variant	Profile	Latency ms/token	Throughput tokens/s	Downlink KB/token
Full proposed framework	GPU	96	10.4	62.1
No speculative decoding	GPU	126	7.9	62.1
No language-adaptive compression	GPU	114	8.8	151.3
No split vocabulary projection	GPU	137	7.3	151.3
No block/asynchronous overlap	CPU	184	5.4	64.2
No endpoint KV-cache control	GPU	97	10.3	62.1

7.8 Sensitivity to Split Dimension

The endpoint-side split dimension D_{end} controls the trade-off between endpoint computation and cloud dependence. Table 6 varies D_{end} while keeping $D = 4096$ and using Chinese-only generation on the GPU endpoint. Very small split dimensions reduce endpoint computation but make the local projection less informative. Very large split dimensions increase endpoint workload and reduce the benefit of offloading. The default value $D_{\text{end}} = 96$ provides the best latency-throughput balance in our setting.

模式下，保留了完整的词汇表，因此大部分逻辑压缩的好处消失了，但推测解码和分割计算仍然会改善延迟。

表 4：不同语言模式下的吞吐量和每个令牌下行链路负载。

Endpoint	Language mode	Throughput tokens/s	Downlink KB/token	Compression vs. full logits
CPU	Chinese-only	6.6 ± 0.2	64.2	56.2%
CPU	English-only	6.9 ± 0.2	49.1	66.5%
CPU	Mixed	5.4 ± 0.1	154.0	0.0%
GPU	Chinese-only	10.4 ± 0.3	62.1	57.6%
GPU	English-only	11.5 ± 0.3	47.8	67.4%
GPU	Mixed	8.6 ± 0.2	151.3	0.0%
Edge	Chinese-only	6.1 ± 0.2	63.8	56.5%
Edge	English-only	6.5 ± 0.2	48.6	66.8%
Edge	Mixed	4.8 ± 0.1	153.1	0.0%

对于 8 位 logits，完整的 150,000 维向量在元数据和身份验证标签之前需要大约 146.5 KB。测量的有效载荷可近似为

$$\text{Payload} \approx |\mathcal{V}_\ell| \cdot b/8 + B_{\text{tag}} + B_{\text{meta}} + B_{\text{pad}}, \quad (21)$$

其中 B_{tag} 、 B_{meta} 和 B_{pad} 表示 AES-GCM 标签、块/会话元数据和填充。因此，测量值略大于理论截断对数大小。

7.7 消融研究

表 5 列出了每个组件的贡献。删除推测性解码会大大增加延迟，因为云必须执行更多经过验证的目标模型解码轮次。去除语言自适应压缩主要是增加下行链路负载。删除分割词汇表投影会增加延迟和有效负载，因为端点必须等待更大的云端日志。消除 CPU 块重叠会损害 CPU 延迟，而消除端点 KV 缓存控制对延迟影响很小，但会削弱隐私控制目标。

表 5：主要框架组件的消融结果。延迟和有效负载越低越好。

Variant	Profile	Latency ms/token	Throughput tokens/s	Downlink KB/token
Full proposed framework	GPU	96	10.4	62.1
No speculative decoding	GPU	126	7.9	62.1
No language-adaptive compression	GPU	114	8.8	151.3
No split vocabulary projection	GPU	137	7.3	151.3
No block/asynchronous overlap	CPU	184	5.4	64.2
No endpoint KV-cache control	GPU	97	10.3	62.1

7.8 对分裂维度的敏感性

端点侧分割维度 D_{end} 控制端点计算和云依赖性之间的权衡。表 6 改变了 D_{end} ，同时保留 $D = 4096$ 并在 GPU 端点上使用纯中文生成。非常小的分割尺寸减少了端点计算，但使局部投影的信息量较少。非常大的分割尺寸会增加端点工作负载并降低卸载的好处。默认值 $D_{\text{end}} = 96$ 在我们的设置中提供了最佳的延迟-吞吐量平衡。

Table 6: Sensitivity analysis for endpoint-side projection dimension on the GPU endpoint.

D_{end}	Endpoint ratio	Latency ms/token	Throughput tokens/s	Observation
48	1.17%	101	9.9	Lowest endpoint cost but weaker local contribution
96	2.34%	96	10.4	Best latency-throughput balance
128	3.12%	95	10.5	Slightly higher local cost with marginal gain
256	6.25%	98	10.2	Useful for stronger local privacy-sensitive projection
512	12.50%	109	9.2	Local projection dominates endpoint latency

7.9 Sensitivity to Speculative Length

Table 7 varies the draft length N on the GPU endpoint. Moderate draft lengths improve latency by reducing cloud interaction rounds. Overly long draft sequences lower the effective acceptance rate and increase verification overhead. The best trade-off appears at $N = 5$.

Table 7: Sensitivity analysis for speculative decoding length on the GPU endpoint.

Draft length N	Acceptance rate	Latency ms/token	Throughput tokens/s
1	96.5%	119	8.4
2	93.7%	107	9.3
4	88.1%	98	10.2
5	85.4%	96	10.4
6	81.2%	99	10.1
8	73.6%	110	9.1

7.10 Generation-Quality Evaluation

Because the cloud decoder remains the authoritative target model and draft tokens are verified before KV-cache update, the proposed method largely preserves the target model’s generation behavior. We evaluate next-token agreement on the 1,000-prompt workload, perplexity on the held-out prompt continuations, MMLU with a 5-shot prompt format, GSM8K with greedy chain-of-thought decoding, and HumanEval pass@1 with deterministic decoding. For task-level evaluation, all compared methods use the same prompt templates, decoding parameters, and evaluation scripts, so the reported differences reflect the collaborative inference mechanism rather than prompt or decoding changes. Table 8 shows that the proposed framework remains close to the full cloud model. The remaining degradation is mainly caused by 8-bit tensor quantization, language-adaptive masking, and endpoint-side approximation in the local projection.

Table 8: Generation-quality comparison with the full cloud model. Higher is better except perplexity.

Method	Next-token agreement	PPL	MMLU	GSM8K	HumanEval pass@1
Full cloud model	100.0%	7.82	54.6	49.8	27.4
Proposed framework	98.6%	7.87	54.3	49.4	27.1
Proposed w/o language masking	98.8%	7.85	54.4	49.5	27.2
Endpoint-only lightweight	84.2%	10.94	46.1	37.8	18.6

7.11 Privacy-Exposure Accounting

Privacy cannot be established by efficiency metrics alone. Table 9 accounts for the information visible to the cloud under different strategies. The proposed framework avoids sending raw prompts, plaintext history identifiers, endpoint-local module weights, and full logits. However, the cloud still observes decrypted feature tensors needed for authorized inference, cache-control metadata, and draft-token verification traces. Therefore, the framework reduces exposure under the stated honest-but-curious threat model rather than providing a formal privacy guarantee.

表6: GPU端点上端点侧投影维度的敏感性分析。

D_{end}	Endpoint ratio	Latency ms/token	Throughput tokens/s	Observation
48	1.17%	101	9.9	Lowest endpoint cost but weaker local contribution
96	2.34%	96	10.4	Best latency-throughput balance
128	3.12%	95	10.5	Slightly higher local cost with marginal gain
256	6.25%	98	10.2	Useful for stronger local privacy-sensitive projection
512	12.50%	109	9.2	Local projection dominates endpoint latency

7.9 对推测长度的敏感性

表 7 改变了 GPU 端点上的草稿长度 N 。适度的草稿长度可以通过减少云交互次数来改善延迟。过长的草稿序列会降低有效接受率并增加验证开销。最佳权衡出现在 $N = 5$ 。

表 7: GPU 端点上推测解码长度的敏感性分析。

Draft length N	Acceptance rate	Latency ms/token	Throughput tokens/s
1	96.5%	119	8.4
2	93.7%	107	9.3
4	88.1%	98	10.2
5	85.4%	96	10.4
6	81.2%	99	10.1
8	73.6%	110	9.1

7.10 发电质量评估

由于云解码器仍然是权威的目标模型，并且草稿令牌在 KV 缓存更新之前经过验证，因此所提出的方法在很大程度上保留了目标模型的生成行为。我们评估 1,000 个提示工作负载的下一个令牌一致性、保留提示延续的困惑度、具有 5 次提示格式的 MMLU、具有贪婪思想链解码的 GSM8K 以及具有确定性解码的 HumanEval pass@1。对于任务级评估，所有比较方法都使用相同的提示模板、解码参数和评估脚本，因此报告的差异反映了协作推理机制，而不是提示或解码的变化。表 8 显示所提出的框架仍然接近完整的云模型。剩余的退化主要是由 8 位张量量化、语言自适应掩蔽和局部投影中的端点侧近似引起的。

表 8: 与完整云模型的发电质量比较。除了困惑之外，越高越好。

Method	Next-token agreement	PPL	MMLU	GSM8K	HumanEval pass@1
Full cloud model	100.0%	7.82	54.6	49.8	27.4
Proposed framework	98.6%	7.87	54.3	49.4	27.1
Proposed w/o language masking	98.8%	7.85	54.4	49.5	27.2
Endpoint-only lightweight	84.2%	10.94	46.1	37.8	18.6

7.11 隐私暴露会计

隐私不能仅通过效率指标来建立。表 9 列出了不同策略下云可见的信息。所提出的框架避免发送原始提示、明文历史标识符、端点本地模块权重和完整日志。然而，云仍然观察授权推理、缓存控制元数据和草稿令牌验证跟踪所需的解密特征张量。因此，该框架减少了在所述诚实但好奇的威胁模型下的暴露，而不是提供正式的隐私保证。

Table 9: Privacy-exposure accounting for different serving strategies.

Method	Raw prompt	Plain history ID	Feature / hidden states	Token trace	Full logits	Endpoint private module	Formal guarantee
Cloud-only	Yes	Yes	No	Yes	No	No	No
Cloud-only encrypted transport	Yes	Yes	No	Yes	No	No	No
Naive split inference	No	Often	Yes	Yes	Often	No	No
Endpoint-only lightweight	No	No	No	No	No	Yes	Local
Proposed framework	No	No	Auth. tensors	Draft / accepted	No	Yes	No

8 Security Scope and Limitations

The proposed framework is designed to reduce privacy exposure in practical edge–cloud deployment, but its security scope should be interpreted carefully. AES-GCM protects transmitted payloads against passive eavesdropping and tampering when keys and nonces are managed correctly, but the authorized cloud still observes decrypted tensors that are necessary for its part of inference. Endpoint-local LoRA modules and draft modules reduce the risk of hidden-state reconstruction by keeping important lightweight weights off the cloud. Endpoint-controlled KV-cache authorization reduces unnecessary historical-context exposure.

However, the framework does not claim formal differential privacy. Hidden states can still carry information about the input, especially against strong adversaries with auxiliary knowledge or model access. Prior work on model inversion, inference attacks, and ML side channels motivates evaluating representation leakage rather than relying only on encrypted transport [49, 50, 51]. Deployment in high-risk environments should therefore combine this architecture with additional measures such as secure enclaves, strict access control, key rotation, audit logging, rate limiting, privacy evaluation against representation-inversion attacks, and stronger alternatives such as secure multi-party computation or differentially private forward passes when their overhead is acceptable [52, 53].

9 Discussion

The design highlights a system-level trade-off between privacy, latency, bandwidth, and model quality. Increasing the endpoint-side dimension D_{end} can reduce dependence on cloud logits but increases endpoint computation. Increasing the speculative length N can reduce interaction rounds when acceptance is high, but may waste computation when draft quality is poor. Reducing L_{Cache} improves privacy control but can degrade contextual coherence. These trade-offs can be configured by endpoint type and application scenario.

The empirical results show that the cache-authorized split-logits design improves latency and communication efficiency while keeping generation quality close to the full cloud model. The most important deployment trade-offs should be interpreted jointly rather than as isolated latency or privacy metrics, since a configuration that minimizes communication may still degrade generation quality or contextual coherence.

10 Conclusion

This paper presents an efficient and privacy-aware edge–cloud collaborative inference framework for LLMs. By combining endpoint-controlled KV-cache authorization, device-adaptive feature preparation, cloud-side decoder inference, split vocabulary projection, language-adaptive logits compression, speculative decoding, AES-GCM encrypted transmission, and endpoint-local LoRA-adapted modules, the framework addresses the practical tension among latency, compute constraints, and privacy protection. The design supports heterogeneous endpoints ranging from pure CPU devices to GPU-equipped clients and ONNX-based edge devices, providing a foundation for deployable privacy-aware LLM inference systems. The evaluation further shows that the proposed framework improves latency and bandwidth efficiency across heterogeneous endpoint profiles while maintaining generation quality close to the full cloud model.

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023.

表 9: 不同服务策略的隐私暴露核算。

Method	Raw prompt	Plain history ID	Feature / hidden states	Token trace	Full logits	Endpoint private module	Formal guarantee
Cloud-only	Yes	Yes	No	Yes	No	No	No
Cloud-only encrypted transport	Yes	Yes	No	Yes	No	No	No
Naive split inference	No	Often	Yes	Yes	Often	No	No
Endpoint-only lightweight	No	No	No	No	No	Yes	Local
Proposed framework	No	No	Auth. tensors	Draft / accepted	No	Yes	No

8 安全范围和限制

所提出的框架旨在减少实际边缘云部署中的隐私暴露，但应仔细解释其安全范围。当正确管理密钥和随机数时，AES-GCM 可以保护传输的有效负载免受被动窃听和篡改，但授权云仍然会观察其推理部分所需的解密张量。端点本地 LoRA 模块和草稿模块通过将重要的轻量级权重保留在云端之外，降低了隐藏状态重建的风险。端点控制的 KV 缓存授权减少了不必要的历史上下文暴露。

然而，该框架并不主张正式的差异隐私。隐藏状态仍然可以携带有关输入的信息，特别是针对具有辅助知识或模型访问权限的强大对手。先前关于模型反转、推理攻击和 ML 侧通道的工作促进了评估表示泄漏，而不是仅依赖于加密传输 [49,50,51]。因此，高风险环境中的部署应将此架构与其他措施相结合，例如安全飞地、严格访问控制、密钥轮换、审计日志记录、速率限制、针对表示反转攻击的隐私评估，以及更强大的替代方案，例如安全多方计算或差分隐私前向传递（当其开销可接受时）[52, 53]。

9 讨论

该设计强调了隐私、延迟、带宽和模型质量之间的系统级权衡。增加端点侧维度 D_{end} 可以减少对云 logits 的依赖，但会增加端点计算。当接受度较高时，增加推测长度 N 可以减少交互轮次，但当草稿质量较差时可能会浪费计算。减少 L_{Cache} 可以改善隐私控制，但会降低上下文的连贯性。这些权衡可以通过端点类型和应用场景进行配置。

实证结果表明，缓存授权的分割逻辑设计提高了延迟和通信效率，同时保持发电质量接近全云模型。最重要的部署权衡应该联合解释，而不是解释为孤立的延迟或隐私指标，因为最小化通信的配置仍可能降低生成质量或上下文一致性。

10 结论

本文为大语言模型提出了一种高效且具有隐私意识的边缘云协作推理框架。通过结合端点控制的 KV 缓存授权、设备自适应特征准备、云端解码器推理、分割词汇投影、语言自适应 logits 压缩、推测解码、AES-GCM 加密传输和端点本地 LoRA 自适应模块，该框架解决了延迟、计算约束和隐私保护之间的实际紧张关系。该设计支持从纯 CPU 设备到配备 GPU 的客户端和基于 ONNX 的边缘设备的异构端点，为可部署的隐私感知 LLM 推理系统提供了基础。评估进一步表明，所提出的框架提高了异构端点配置文件的延迟和带宽效率，同时保持接近完整云模型的生成质量。

参考

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Codyhao Yu, Joseph E. Gonzalez, Hao Zhang 和 Ion Stoica。通过 pagedattention 服务的大语言模型的高效内存管理。在 *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023 年。

- [2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- [3] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, 2024.
- [4] Gyeong-In Yu, Jeongseok Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. ORCA: A distributed serving system for transformer-based generative models. In *USENIX Symposium on Operating Systems Design and Implementation*, 2022.
- [5] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Conglong Luo, Arash Elnawany, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022.
- [6] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017.
- [7] Yoshitaka Matsubara, Marco Levorato, and Francesco Restuccia. Split computing and early exiting for deep learning applications: Survey and research challenges. *ACM Computing Surveys*, 2022.
- [8] Hao Liang, Qiyang Sang, Chuan Hu, Dapeng Cheng, et al. DNN surgery: Accelerating DNN inference on the edge through layer partitioning. *IEEE Transactions on Parallel and Distributed Systems*, 2023.
- [9] Yinan Hu, Connor Imes, Xiaoang Zhao, Suhas Kundu, et al. Pipeedge: Pipeline parallelism for large-scale model inference on heterogeneous edge devices. In *Euromicro Conference on Digital System Design / Software Engineering and Advanced Applications*, 2022.
- [10] Zeyu Hao, Hao Jiang, Shaoxiang Jiang, Jiajun Ren, and Ting Cao. Hybrid SLM and LLM for edge-cloud collaborative inference. In *ACM MobiFoundation Workshop*, 2024.
- [11] Sen Ye, Bin Ouyang, Laizhong Zeng, Tian Qian, Xiangjie Chu, et al. Jupiter: Fast and resource-efficient collaborative inference of generative LLMs on edge devices. In *IEEE INFOCOM*, 2025.
- [12] Jiayi Ning, Chenning Zheng, and Tong Yang. DSSD: Efficient edge-device LLM deployment and collaborative inference via distributed split speculative decoding. *arXiv preprint arXiv:2507.12000*, 2025.
- [13] Haojie Jin and Yufei Wu. CE-CoLLM: Efficient and adaptive large language models through cloud-edge collaboration. In *IEEE International Conference on Web Services*, 2025.
- [14] Mingyuan Zhang, Xiang Shen, Jiannong Cao, Zhenyu Cui, et al. Edgeshard: Efficient LLM inference via collaborative edge computing. *IEEE Internet of Things Journal*, 2024.
- [15] Shuai Li, Hao Wang, Wenkang Xu, Rongxin Zhang, Song Guo, Jingdong Yuan, et al. Collaborative inference and learning between edge SLMs and cloud LLMs: A survey. *arXiv preprint arXiv:2507.16731*, 2025.
- [16] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023.
- [17] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023.
- [18] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rui Chen, and Zhihao Jia. Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. In *ACM SIGOPS Symposium on Operating Systems Principles*, 2024.
- [19] Zhenyu Chen, Avner May, Roman Svirschevski, Yuhuang Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding. In *Advances in Neural Information Processing Systems*, 2024.
- [20] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning*, 2024.
- [21] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Conference on Empirical Methods in Natural Language Processing*, 2024.
- [22] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding. *arXiv preprint*, 2024.

[2] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra 和 Christopher Ré. Flashattention: 具有 IO 感知能力的快速、内存高效的精确注意力。在 *Advances in Neural Information Processing Systems*, 2022 年。[3] Tri Dao。Flashattention-2: 更快的注意力, 更好的并行性和工作分区。 *International Conference on Learning Representations*, 2024 年。[4] Gyeong-In Yu, Jeongseok Jeong, Geon-Woo Kim, Soojeong Kim 和 Byung-Gon Chun。ORCA: 基于变压器的生成模型的分布式服务系统。 *USENIX Symposium on Operating Systems Design and Implementation*, 2022 年。[5] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Conglong Luo, Arash Elnawany, Shaden Smith, Minjia Zhang, Jeff Rasley 和 Yuxiong He。DeepSpeed-Inference: 以前所未有的规模实现变压器模型的高效推理。 *International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022 年。[6] Yiping Kang, Johann Hauswald, Cao Gau, Austin Rovinski, Trevor Mudge, Jason Mars 和 Lingjia Tang。神经外科医生: 云和移动边缘之间的协作智能。 *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017 年。[7] Yoshitaka Matsubara, Marco Levorato 和 Francesco Restuccia。深度学习应用的分割计算和早期退出: 调查和研究挑战。 *ACM Computing Surveys*, 2022。[8] 梁浩, 桑启阳, 胡传, 程大鹏, 等。DNN 手术: 通过层划分加速边缘的 DNN 推理。 *IEEE Transactions on Parallel and Distributed Systems*, 2023。[9] Yinan Hu, Connor Imes, Xiaolang Zhu, Suhas Kundu, 等。Pipeedge: 异构边缘设备上大规模模型推理的管道并行性。 *Euromicro Conference on Digital System Design / Software Engineering and Advanced Applications*, 2022 年。[10] 郝泽宇, 蒋浩, 蒋少翔, 任家军, 曹廷。用于边缘云协作推理的混合 SLM 和 LLM。 *ACM MobiFoundation Workshop*, 2024。[11] 叶森, 欧阳斌, 曾来忠, 田谦, 褚向杰, 等。Jupiter: 边缘设备上生成 LLM 的快速且资源高效的协作推理。在 *IEEE INFOCOM*, 2025 年。

[12] 宁嘉怡, 郑辰宁, 杨童。DSSD: 通过分布式分割推测解码进行高效的边缘设备 LLM 部署和协作推理。 *arXiv preprint arXiv:2507.12000*, 2025 年。

[13] 金浩杰, 吴宇飞。CE-CoLLM: 通过云边缘协作实现高效且自适应的大语言模型。

IEEE International Conference on Web Services, 2025。[14] 张明远, 沉翔, 曹建农, 崔振宇, 等。Edgeshard: 通过协作边缘计算进行高效的 LLM 推理。 *IEEE Internet of Things Journal*, 2024。[15] 李帅, 王浩, 徐文康, 张荣鑫, 郭松, 袁敬东, 等。边缘 SLM 和云 LLM 之间的协作推理和学习: 一项调查。

arXiv preprint arXiv:2507.16731, 2025。[16] Yaniv Leviathan, Matan Kalman 和 Yossi Matias。通过推测解码从 Transformer 进行快速推理。 *International Conference on Machine Learning*, 2023 年。[17] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre 和 John Jumper。通过推测性采样加速大语言模型解码, 2023 年。[18] Xupeng Miao, Gabriele Oliaro, Zhihao Zhu, Xinhao Cheng, Zeyu Wang, Rui Chen 和 zhihao Jia。Specinfer: 通过基于树的推测推理和验证来加速生成大语言模型。

ACM SIGOPS Symposium on Operating Systems Principles, 2024 年。[19] 陈振宇, Avner May, Roman Svirskiy, Yuhuang Huang, Max Ryabinin, Zhihao Jia 和 Beidi Chen。Sequoia: 可扩展、强大且硬件感知的推测解码。 *Advances in Neural Information Processing Systems*, 2024 年。[20] 蔡天乐, 李宇红, 耿正阳, 彭洪武, Jason D. Lee, 陈德明, Tri Dao。Medusa: 具有多个解码头的简单 LLM 推理加速框架。

International Conference on Machine Learning, 2024 年。[21] 李宇辉, 魏方云, 张超, 张红阳。EAGLE-2: 使用动态草稿树更快地推理语言模型。 *Conference on Empirical Methods in Natural Language Processing*, 2024 年。[22] 付一超, Peter Bailis, Ion Stoica, 张浩。使用前瞻解码打破 LLM 推理的顺序依赖性。 *arXiv preprint*, 2024 年。

- [23] Yingqian Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Sashank J. Reddi, and Felix Yu. Distillspec: Improving speculative decoding via knowledge distillation. In *International Conference on Learning Representations*, 2024.
- [24] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, et al. Layerskip: Enabling early exit inference and self-speculative decoding. In *Annual Meeting of the Association for Computational Linguistics*, 2024.
- [25] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In *Annual Meeting of the Association for Computational Linguistics*, 2024.
- [26] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [27] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, 2023.
- [28] Zhiwei Lin, Xinyi Hu, Yifan Zhang, Zhenyu Chen, Zihao Fang, Xiang Chen, et al. Splitlora: A split parameter-efficient fine-tuning framework for large language models. *arXiv preprint arXiv:2407.00952*, 2024.
- [29] Yansong Sun, Zhenyu Li, Yue Li, and Bolin Ding. Improving LoRA in privacy-preserving federated learning. *arXiv preprint arXiv:2403.12313*, 2024.
- [30] Shima Malekmohammadi and Golnoosh Farnadi. On the implicit relation between low-rank adaptation and differential privacy. In *NeurIPS Workshop on Privacy in Machine Learning*, 2024.
- [31] Dongqi Yao and Bin Li. Is split learning privacy-preserving for fine-tuning large language models? *IEEE Transactions on Big Data*, 2024.
- [32] Yicheng Huang, Jianlong Yin, Zihao Tan, Qiang Wang, et al. SLDP-LoRA: A privacy-preserving split learning framework with low-rank adaptation. In *IEEE International Conference on Network Security and Engineering*, 2025.
- [33] Pinghui Mai, Rui Yan, Zeyu Huang, Yifei Yang, and Yanmin Pang. Split-and-denoise: Protect large language model inference with local differential privacy. *arXiv preprint arXiv:2310.09130*, 2023.
- [34] Muslum Ugurbil, Dimitris Mouris, Miguel B. Santos, et al. Fission: Distributed privacy-preserving large language model inference. In *IEEE European Symposium on Security and Privacy*, 2025.
- [35] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.
- [36] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Guanxiao Xiao, et al. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Conference on Machine Learning and Systems*, 2024.
- [37] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, 2023.
- [38] Mingjie Sun, Zhiyuan Liu, Anna Bair, and Zico Kolter. A simple and effective pruning approach for large language models. In *International Conference on Learning Representations*, 2024.
- [39] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. In *International Conference on Learning Representations*, 2024.
- [40] Hao Shen, Hui Chang, Bo Dong, Yue Luo, and Hao Meng. Efficient LLM inference on CPUs. In *Efficient Deep Learning: Methods, Systems, and Applications*. Springer, 2025.
- [41] Zhenyu Zhang, Yao Sheng, Tianyi Jin, Chi Chen, Tianqi Liang, et al. H2O: Heavy-hitter oracle for accurate KV cache compression. In *Advances in Neural Information Processing Systems*, 2023.
- [42] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Streamingllm: Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.
- [43] National Institute of Standards and Technology. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication 800-38D, 2007.
- [44] Qipeng Pang, Jingyi Zhu, Hendrik Möllering, Wenqiang Zheng, et al. BOLT: Privacy-preserving, accurate and efficient inference for transformers. In *IEEE Symposium on Security and Privacy*, 2024.

[23] 周英前, 吕凯峰, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Sashank J. Reddi, Felix Yu. Distillspec: 通过知识蒸馏改进推测解码。 *International Conference on Learning Representations*, 2024 年。 [24] Mostafa Elhoushi、Akshat Shrivastava、Diana Liskovich、Basil Hosmer、Bram Wasti、Liangzhen Lai、Anas Mahmoud、Bilge Acun、Saurabh Agarwal、Ahmed Roman 等。Layerskip: 启用早期退出推理和自推测解码。 *Annual Meeting of the Association for Computational Linguistics*, 2024 年。 [25] 张军, 王珏, 李欢, 寿力丹, 陈科, 陈刚, Sharad Mehrotra. 起草和验证: 通过自推测解码进行无损大语言模型加速。 *Annual Meeting of the Association for Computational Linguistics*, 2024 年。 [26] Edward J. Hu、Yelong Shen、Phillip Wallis、Zeyuan Allen-Zhu、Yuanzhi Li、Shean Wang、Lu Wang 和 Weizhu Chen. Lora: 大语言模型的低阶适应。 *International Conference on Learning Representations*, 2022 年。 [27] Tim Dettmers、Artidoro Pagnoni、Ari Holtzman 和 Luke Zettlemoyer. QLoRA: 量化 LLM 的高效微调。 *Advances in Neural Information Processing Systems*, 2023。 [28] 林志伟, 胡欣怡, 张一凡, 陈振宇, 方子浩, 陈翔, 等。Splitlora: 用于大语言模型的分割参数高效微调框架。 *arXiv preprint arXiv:2407.00952*, 2024。 [29] 孙岩松, 李振宇, 李悦, 丁柏林. 改进 LoRA 在隐私保护联邦学习中的应用。 *arXiv preprint arXiv:2403.12313*, 2024。 [30] Shima Malekmohammadi 和 Golnoosh Farnadi. 论低秩适应与差分隐私之间的隐含关系。 *NeurIPS Workshop on Privacy in Machine Learning*, 2024。 [31] 姚东奇, 李斌. 分割学习是否可以保护隐私以微调大语言模型? *IEEE Transactions on Big Data*, 2024。 [32] 黄一成, 尹建龙, 谭子豪, 王强, 等。SLDP-LoRA: 一种具有低秩适应的隐私保护分割学习框架。 *IEEE International Conference on Network Security and Engineering*, 2025。 [33] 麦平辉, 严睿, 黄泽宇, 杨一飞, 庞彦民. 分割和降噪: 通过本地差分隐私保护大语言模型推理。 *arXiv preprint arXiv:2310.09130*, 2023。 [34] Muslum Ugurbil、Dimitris Mouris、Miguel B. Santos 等人。Fission: 分布式隐私保护大语言模型推理。 *IEEE European Symposium on Security and Privacy*, 2025 年。 [35] Elias Frantar、Saleh Ashkboos、Torsten Hoefler 和 Dan Alistarh. GPTQ: 生成预训练 Transformer 的准确训练后量化。 *International Conference on Learning Representations*, 2023。 [36] 吉林, 唐家明, 唐浩天, 尚阳, 肖关晓, 等。AWQ: 用于设备上 LLM 压缩和加速的激活感知权重量化。 *Conference on Machine Learning and Systems*, 2024 年。 [37] Guangxuan Shaw、Ji Lin、Mickaël Seznec、Hao Wu、Julien Demouth 和 Song Han. Smoothquant: 大语言模型的准确高效的训练后量化。 *International Conference on Machine Learning*, 2023 年。 [38] 孙明杰, 刘志远, 安娜·贝尔, 和齐科·科尔特. 一种简单有效的大语言模型修剪方法。 *International Conference on Learning Representations*, 2024 年。 [39] Yuxian Gu, Li Dong, Furu Wei, Minlie Huang. Minillm: 大语言模型的知识蒸馏。 *International Conference on Learning Representations*, 2024 年。 [40] 沉浩, 张辉, 董波, 罗越, 孟浩. CPU 上的高效 LLM 推理。在 *Efficient Deep Learning: Methods, Systems, and Applications* 中。Springer, 2025。 [41] 张振宇, 盛耀, 金天一, 陈驰, 梁天琪, 等。H2O: 用于精确 KV 缓存压缩的重量级预言机。 *Advances in Neural Information Processing Systems*, 2023。 [42] Guangxuan Shaw, Yuandong Tian, Beidi Chen, Song Han, 和 Mike Lewis. Streamingllm: 具有注意力接收器的高效流式语言模型。 *International Conference on Learning Representations*, 2024 年。 [43] 美国国家标准与技术研究院. 分组密码操作模式的建议: 伽罗瓦/计数器模式 (gcm) 和 gmac。NIST 特别出版物 800-38D, 2007。 [44] Qipeng Pang, Jingyi Zhu, Hendrik Möllering, Wenqiang Cheng, 等。BOLT: 保护隐私、准确高效的变压器推理。在 *IEEE Symposium on Security and Privacy*, 2024 年。

- [45] Tianyu Chen, Hang Bao, Shaohan Huang, Li Dong, Binxing Jiao, et al. THE-X: Privacy-preserving transformer inference with homomorphic encryption. In *Findings of the Association for Computational Linguistics*, 2022.
- [46] Dacheng Li, Rulin Shao, Hongyi Wang, Han Guo, Eric P. Xing, et al. MPCFormer: Fast, performant and private transformer inference with MPC. *arXiv preprint*, 2022.
- [47] Tashin Nayan, Zhenkai Zhang, and Ruizhe Sun. Secureinfer: Heterogeneous TEE-GPU architecture for privacy-critical LLM tensors. *arXiv preprint arXiv:2510.19979*, 2025.
- [48] Microsoft. ONNX Runtime: Cross-platform, high performance machine learning inferencing and training accelerator. <https://onnxruntime.ai/>. Accessed 2026.
- [49] Ruiyi Zhang, Shuichiro Hidano, and Farinaz Koushanfar. Text revealer: Private text reconstruction via model inversion attacks against transformers. *arXiv preprint arXiv:2209.10505*, 2022.
- [50] Yugeng Liu, Rui Wen, Xiang He, Ahmed Salem, Zhikun Zhang, et al. ML-Doctor: Holistic risk assessment of inference attacks against machine learning models. In *USENIX Security Symposium*, 2022.
- [51] Edoardo Debenedetti, Giorgio Severi, Nicholas Carlini, et al. Privacy side channels in machine learning systems. In *USENIX Security Symposium*, 2024.
- [52] Irene Zhou, Farbod Tofigh, Massimo Piccardi, Mehran Abolhasan, et al. Secure multi-party computation for machine learning: A survey. *IEEE Communications Surveys & Tutorials*, 2024.
- [53] Min Du, Xiang Yue, Sherman S. M. Chow, Tianhao Wang, Chenguang Huang, et al. DP-Forward: Fine-tuning and inference on language models with differential privacy in forward pass. In *ACM SIGSAC Conference on Computer and Communications Security*, 2023.

[45] 陈天宇, 鲍航, 黄少涵, 李东, 焦滨兴, 等。THE-X: 使用同态加密的隐私保护变压器推理。 *Findings of the Association for Computational Linguistics*, 2022。[46] 李大成, 邵儒林, 王宏毅, 韩国, Eric P. Xing, 等。MPCFormer: 使用 MPC 进行快速、高性能且私有的变压器推理。 *arXiv preprint*, 2022。[47]Tashin Nayan, 张振凯, 孙瑞哲。Secureinfer: 用于隐私关键 LLM 张量的异构 TEE-GPU 架构。 *arXiv preprint arXiv:2510.19979*, 2025。[48]微软。ONNX Runtime: 跨平台、高性能机器学习推理和训练加速器。 <https://onnxruntime.ai/>。2026 年访问。[49] Ruiyi 张、Shuichiro Hidano 和 Farinaz Koushanfar。文本揭示者: 通过针对 Transformer 的模型反转攻击来重建私人文本。 *arXiv preprint arXiv:2209.10505*, 2022。[50] 刘玉庚, 文瑞, 何翔, Ahmed Salem, 张志坤, 等。ML-Doctor: 针对机器学习模型的推理攻击的整体风险评估。 *USENIX Security Symposium*, 2022 年。[51] Edoardo Debenedetti、Giorgio Severi、Nicholas Carlini 等人。机器学习系统中的隐私侧通道。 *USENIX Security Symposium*, 2024 年。[52] Irene Zhou、Farbod Tofigh、Massimo Piccardi、Mehran Abolhasan 等人。机器学习的安全多方计算: 一项调查。 *IEEE Communications Surveys & Tutorials*, 2024。[53]杜敏, 岳翔, Sherman S.M.Chow, 王天浩, 黄晨光, 等。DP-Forward: 在前向传播中对具有差异隐私的语言模型进行微调和推理。在 *ACM SIGSAC Conference on Computer and Communications Security*, 2023 年。